

CRUD met relaties uitleg

Entiteiten en Attributen

Wat is een entiteit?

Een **entiteit** is een “ding” waar je gegevens van wilt opslaan.

voorbeelden zijn:

- Klant
- Abonnement
- Categorie
- SubCategorie
- MenuKaartItem
- Student
- Project

In de database wordt een entiteit een tabel.

Entiteiten hebben een **attribuut** ook, dit zijn eigenschappen / gegevens van de entiteit.

Een klant heeft bijvoorbeeld de volgende attributen:

- ID
- Naam
- Email
- Telefoonnummer

Simpel gezegd, de entiteit is de tabel en de attribuut is de kolom.

Keys

De tabellen (entiteiten) hebben ook verschillende soorten keys, of naja eigenlijk zijn het de attributen die keys kunnen zijn.

De Primary key:

Dit gebruik je om een rij uniek te maken.

Dit is meestal een ID, een klant heeft een ID deze moet uniek zijn dus dit is de **primary key**.

De Foreign Key:

Een **foreign key** is een verwijzing naar de primary key van een andere tabel.

Stel je hebt een klas tabel met KlasID en een student tabel met StudentID.

Een student zit in een klas dus die tabel van student heeft een verwijzing naar klas. Dus in de student tabel zit ook KlasID en dit is de **foreign key**.

De Alternate Key:

Een **alternate key** is ook uniek, maar het is niet de primary key.

Stel de student entiteit heeft een attribuut genaamd Emailadres. Een emailadres moet ook uniek zijn maar is niet de primary key. Dit is de **Alternate key**. StudentID is het technische ID. Maar emailadres moet ook uniek zijn, want twee studenten mogen niet hetzelfde emailadres hebben.

Relaties tussen tabellen

Wat is een relatie eigenlijk?

Een relatie betekent tabel A heeft iets te maken met tabel B.

bijvoorbeeld:

- Klas heeft studenten
- categorie heeft subcategorie
- klas heeft modules

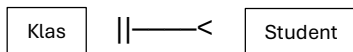
In sql maken we die relaties met de foreign key.

Om de tabellen in bijvoorbeeld visio te verbinden met elkaar gebruiken we lijnen. Er kunnen 2 dingen staan aan t uiteinde van zo'n lijn.

|| = precies 1

< = veel

stel je hebt dit:



Dan betekent dit:

1 klas heeft meerdere studenten

Meerdere studenten horen bij 1 klas

Onthoud:

De < is de veel-kant

en de foreign key komt meestal aan de veel-kant.

dus de student tabel kan er zo uitzien:

Student	
StudentID	Pk
Naam	
emailadres	AK
KlasID	FK

Soorten relaties tussen tabellen

Er zijn verschillende soorten relaties tussen tabellen:

- **Geen relatie**
De tabel staat los en verwijst niet naar een andere tabel.
- **1-op-veel** 
1 klas kan meerdere studenten hebben. Maar 1 student hoort bij 1 klas.
- **1-op-veel vanaf de andere kant bekeken** 
1 student hoort bij 1 klas. Maar 1 klas kan meerdere studenten hebben.
- **1-op-1** 
1 persoon heeft 1 paspoort. En 1 paspoort hoort bij 1 persoon.
- **Veel-op-veel** 
1 klant kan meerdere producten bestellen. En 1 product kan door meerdere klanten besteld worden.

Hoe kom je erachter welke relaties de tabellen hebben met elkaar?

Vraag 1:

Kan 1A meerdere B hebben?

vraag 2:

Kan 1B meerdere A hebben?

Nee + nee = 1-op-1

Ja + nee = 1-op-veel

Nee + ja = 1-op-veel vanaf de andere kant

Ja + ja = veel-op-veel

of **methode 2** - Je gaat zinnen maken voor de tabellen.

1 op 1 (persoon - paspoort)

kan persoon 1 paspoort 1 hebben? Ja

Kan persoon 1 paspoort 2 hebben? Nee

kan persoon 2 paspoort 1 hebben? Nee

dus persoon 1 heeft paspoort 1 **maar** paspoort 1 hoort maar bij persoon 1

persoon ||---|| paspoort

1 op veel (Klas - student)

kan klas 1 student 1 hebben? Ja

kan klas 1 student 2 hebben? Ja

kan student 1 bij klas 2 zitten? Nee

dus klas 1 kan meerdere studenten hebben **maar** 1 student hoort maar bij 1 klas

Klas ||---< student

Veel op veel (student - vak)

Kan student 1 vak 1 volgen? Ja

kan student 1 vak 2 volgen? Ja

kan student 2 vak 1 volgen? Ja

Dus student 1 kan meerdere vakken volgen en vak 1 kan door meerdere studenten gevolgd worden

Student >---< vak

Onthoud wel: Je hebt bij een veel op veel een koppeltabel nodig want het werkt niet met 2 tabellen, in dit geval zou je dus een tabel nieuwe tabel krijgen genaamd **studentVak**. Dit komt omdat je anders niet netjes kan opslaan welke student bij welk vak hoort.

Want je kunt niet gewoon **VakID** in Student tabel zetten, want 1 student kan meerdere vakken hebben.

En je kunt **StudentID** ook niet gewoon in Vak tabel zetten, want één vak kan meerdere studenten hebben.

Dus dan krijg je een koppel tabel genaamd **StudentenVak** met daarin: StudentVakID, StudentID, VakID